WEEK 01 - LOGIC

CLASS 04 - FUNCTIONS

EX10 - Rocket Equation as a Function

Multiple parameters, default parameters, and reuse of an engineering function

1. Where we are

We remain in week_01_logic -> class_04_functions. We are not changing week or class. EX09 introduced def, parameters, arguments, return, scope, and reuse. EX10 extends those ideas to a function with several parameters and a physical constant with a default value.

2. Physical model: Tsiolkovsky rocket equation

For an ideal rocket, neglecting gravity, aerodynamic, and steering losses, the ideal delta-v capability is:

$$\Delta v = I_{sp} \cdot g_0 \cdot \ln\left(\frac{m_0}{m_f}\right)$$

Isp: specific impulse [s]
g0: standard gravity [m/s^2]
m0: initial mass [kg]
mf: final mass [kg]

3. From equation to function

The function should behave like a black box: it receives the required physical data, validates them, computes the mass ratio, and returns delta-v. The main program should not repeat the formula.

```
def calculate_rocket_delta_v(isp, initial_mass, final_mass, g0=9.80665):
    ...
    return delta_v
```

4. New concept: default parameter

In EX02 we treated g0 as a known constant. In a reusable function we can make it a parameter with a default value. The normal call does not need to repeat 9.80665, but the caller may still override it for a specific experiment.

```
delta_v = calculate_rocket_delta_v(250, 500000, 50000)

# g0 = 9.80665 is used automatically

delta_v = calculate_rocket_delta_v(250, 500000, 50000, g0=9.81)

# Overrides the default value
```

Important syntax rule

Parameters without default values must appear before parameters with default values.

```
# Correct
def f(a, b, c=10):
```

```
    ...

# Incorrect
def f(a, b=10, c):
    ...
```

5. Parameters vs positional and keyword arguments

A function can be called positionally or by naming the arguments. Keyword arguments are especially useful in engineering functions because they make calls easier to read when several physical parameters are involved.

```
# Positional
calculate_rocket_delta_v(250, 500000, 50000)

# Keyword arguments
calculate_rocket_delta_v(
    isp=250,
    initial_mass=500000,
    final_mass=50000
)
```

6. Validation

For this exercise we keep our previous decision: Isp = 0 is accepted and produces delta-v = 0. Both masses must be strictly positive, and initial mass cannot be lower than final mass. If m0 = mf, the logarithm is zero and delta-v is zero.

Condition	Interpretation
isp < 0	Invalid
initial_mass <= 0	Invalid
final_mass <= 0	Invalid
initial_mass < final_mass	Invalid
initial_mass == final_mass	Valid: delta-v = 0

As in EX09, you can use return None to stop the function for invalid input. Avoid sys.exit() inside a reusable function.

7. Expected algorithm

```
INPUTS
  isp
  initial_mass
  final_mass
  g0 = 9.80665 (default)

VALIDATE INPUTS
  invalid? -> return None

CALCULATE
  mass_ratio = initial_mass / final_mass
  delta_v = isp * g0 * ln(mass_ratio)

RETURN
  delta_v
```

8. What I do NOT want yet

Do not use classes, external constant files, or raise-based exceptions yet. Do not move the function into a separate module either. The goal is to practice a function with several parameters and one default parameter.

9. Test cases

Case	Values	Expected
Nominal	Isp=250, m0=500000, mf=50000	Positive delta-v
No impulse	Isp=0	0 m/s
No propellant use	m0=mf	0 m/s
Invalid mass	mf=0	None
Invalid ratio	m0<mf	None
Invalid Isp	Isp<0	None

10. Reuse

Once the function works, call it several times without repeating the equation. Try one positional call and one call using keyword arguments.

```
dv1 = calculate_rocket_delta_v(250, 500000, 50000)
dv2 = calculate_rocket_delta_v(isp=320, initial_mass=120000, final_mass=30000)
dv3 = calculate_rocket_delta_v(300, 100000, 100000)
```

11. File structure

```
# Imports
from math import log

# Functions
def calculate_rocket_delta_v(...):
    ...

# Main inputs / calls
isp = ...
initial_mass = ...
final_mass = ...

delta_v = calculate_rocket_delta_v(...)

# Output
if delta_v is not None:
    print(...)
```

12. Learning Tracker

Concept	EX10
Function with several parameters	YES
Default parameter	YES
Positional arguments	YES
Keyword arguments	YES

return None as guard clause	YES
Derived variables inside the function	YES
Reuse of an engineering function	YES

Final objective: the main program should be able to request a delta-v through a clear, reusable call without knowing the internal details of the calculation.